

Two-Machine Flow Shop Scheduling to Minimize Total Weighted Completion Time: Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

Minimizing the total weighted completion time on a two-machine flow shop, denoted $F2 \parallel \sum w_j C_j$, is a classical NP-hard scheduling problem with applications in manufacturing, computing, and logistics. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. Here we partition the job set into at most K categories by non-increasing weight, which decomposes the schedule into K sequential blocks. We establish four structural properties of optimal schedules: the permutation property, the positional constraint $\pi(J_k) \leq k$, the block-end condition requiring the final job of each block to carry that block's weight class, and the SPT(1)-LPT(2) (Johnson) ordering rule within each block. Building on these properties and the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a pseudo-polynomial time dynamic programming algorithm that constructs the schedule block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; Total weighted completion time; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Problem Formulation

3 Structural Properties

We partition the job set J into K weight categories $J_1, J_2, \dots, J_K$ such that all jobs in J_k share the same weight W_k , with $W_1 > W_2 > \dots > W_K$. Let $n_k = |J_k|$ and $\sum_{k=1}^K n_k = n$.

The schedule is divided into K contiguous logical segments (blocks) $S_1, S_2, \dots, S_K$, processed sequentially on both machines. We denote by $\pi(j)$ the block index to which job j is assigned.

Lemma 3.1 (*Permutation property.*) *For $F2 \parallel \sum w_j C_j$, there exists an optimal schedule in which the job sequence is identical on both machines.*

This is a classic result for two-machine flow shops with regular objective functions.

Lemma 3.2 (*Positional constraint.*) *In any optimal schedule, $\pi(J_k) \leq k$ for all $k = 1, \dots, K$. Equivalently, every job from weight class J_k is assigned to one of the first k blocks $S_1, S_2, \dots, S_k$.*

Lemma 3.2 implies that once block S_k has been processed, all jobs carrying weights $W_1, \dots, W_k$ are fully scheduled. Consequently, block S_k may contain only jobs from classes $J_k, J_{k+1}, \dots, J_K$.

Lemma 3.3 (*Block-end condition.*) *In any optimal schedule, the final job of block S_k belongs to weight class J_k , for each $k = 1, \dots, K$.*

Lemma 3.4 (*Intra-block ordering.*) *Within each block S_k , all jobs are sequenced according to the SPT(1)-LPT(2) rule (Johnson's rule): for any two jobs $x, y \in S_k$, job x precedes job y if and only if*

$$\min\{a_x, b_y\} < \min\{a_y, b_x\},$$

with ties broken by non-decreasing a_j .

Taken together, these four properties determine the structure of any optimal schedule: jobs are organised into K blocks ordered by decreasing weight; within each block the sequence follows Johnson's rule; class k jobs cannot appear beyond block k ; and the final position of block k is occupied by a job of class k .

4 Dynamic Programming Algorithm

The structural properties established in Section 3 enable a dynamic programming algorithm that builds the schedule block by block. Our approach adapts the interval-based state representation of , originally devised for the $F2|r_j|C_{\max}$ PTAS, to the weighted completion time objective. In that framework, the schedule is partitioned into intervals, each interval is internally sequenced by Johnson’s rule, and the DP state tracks per-interval completion times. We retain the block-structured state space but replace the makespan accumulation with an incremental weighted sum computation.

Notation.	J	全体工件集合
	n	工件总数 ($n = J $)
	K	不同权重的类别数
	J_k	权重为 W_k 的工件集合 ($k = 1, \dots, K$)
	W_k	J_k 中工件的公共权重, 满足 $W_1 > W_2 > \dots > W_K$
	n_k	$ J_k $, 第 k 类工件的数量
	a_j	工件 j 在机器 1 上的加工时间
	b_j	工件 j 在机器 2 上的加工时间
	w_j	工件 j 的权重 ($j \in J_k \iff w_j = W_k$)
	L_k	J_k 中工件按 Johnson 规则排序后的有序列表
	S_k	调度中第 k 个连续块
	$J_{i,k}$	第 k 个块中的第 i 个工件 ($i = 1, \dots, S_k $)
	$\mathcal{Y}^{(k)}$	处理完前 k 个块后的非支配状态集合 ($k = 0, \dots, K$)
	A	前 k 个块在机器 1 上累计加工时间
	B	第 k 个块在机器 2 上的完工时间 (即第 $k+1$ 个块开始时机器 2 可用时刻)
	z	前 k 个块的累计加权完工时间和
	$A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}}$	块构建过程中机器 1 累计时间、机器 2 完工时间、加权成本的运行值
	Z^*	最优 (最小) 总加权完工时间

4.1 Preprocessing

We first partition the n jobs into K sets $J_1, J_2, \dots, J_K$ by weight, with $W_1 > W_2 > \dots > W_K$. Within each set J_k , we sort the jobs according to Johnson’s rule (SPT(1)-LPT(2)). Let the resulting ordered list be

$$L_k = (j_{k,1}, j_{k,2}, \dots, j_{k,n_k}), \quad k = 1, 2, \dots, K,$$

where $n_k = |J_k|$. Since block S_k will contain jobs drawn from classes $k, k+1, \dots, K$ and must obey Johnson ordering internally (Lemma 3.4), the correct within-block sequence is obtained by taking prefixes of each L_m ($m \geq k$) and merging them under the same Johnson comparator. To merge a set of jobs into Johnson order, one partitions it into those with $a_j \leq b_j$ (sorted by non-decreasing a_j) and those with $a_j > b_j$ (sorted by non-increasing b_j), and concatenates the two subsequences. This procedure is used whenever the algorithm merges several L_m suffixes into a single sequence.

4.2 Dynamic Programming Algorithm

We build the schedule block by block. After the first k blocks are fixed, let

$$S_m = (J_{1,m}, J_{2,m}, \dots, J_{|S_m|,m}), \quad m = 1, \dots, k,$$

where $J_{i,m}$ denotes the i -th job in block S_m . Each pre-sorted list L_m is the Johnson order of class J_m . Since blocks consume prefixes of each L_m (Lemma 3.4), the jobs already assigned to $S_1 \cup \dots \cup S_k$ constitute a prefix of each L_m . Consequently, the set of remaining (unassigned) jobs of class J_m is simply the suffix of L_m after that prefix.

A state in $\mathcal{Y}^{(k)}$ is a tuple $(J_{i,k}, A, B, z)$, where $J_{i,k}$ collectively denotes the jobs assigned to each block $S_1, \dots, S_k$ (i.e. $S_m = (J_{1,m}, \dots, J_{|S_m|,m})$ for $m \leq k$), A is the total M1 processing time over blocks $1..k$, B is the M2 completion time at the end of block k , and z is the accumulated weighted completion time. Since blocks consume prefixes of each L_m (Lemma 3.4), the jobs assigned to $S_1 \cup \dots \cup S_k$ constitute a prefix of each L_m ; consequently, the set of remaining (unassigned) jobs of class J_m is simply the suffix of L_m after that prefix. The initial state is $\mathcal{Y}^{(0)} = \{(\emptyset, 0, 0, 0)\}$ with all blocks empty.

To construct block S_{k+1} from a state in $\mathcal{Y}^{(k)}$, we collect the remaining suffixes of $L_{k+1}, \dots, L_K$ and merge them into a single Johnson-ordered sequence. Scanning this sequence from left to right, for each job $j \in \bigcup_{m=k+1}^K J_m$ we decide whether to **include** it in S_{k+1} or **postpone** it to a later block:

- If $j \in J_{k+1}$: **must include** (Lemma 3.2).
- If $j \in J_m$ with $m > k+1$: either include or postpone (binary branch).

When a job is included, it becomes the next $J_{i,k+1}$ and we update:

$$\begin{aligned} A_{\text{cur}} &\leftarrow A_{\text{cur}} + a_j, \\ B_{\text{cur}} &\leftarrow \max(B_{\text{cur}}, A_{\text{cur}}) + b_j, \\ z_{\text{cur}} &\leftarrow z_{\text{cur}} + w_j \cdot B_{\text{cur}}. \end{aligned}$$

A postponed job remains in its suffix and participates in the next block's scan. At the end, we require that the last job of S_{k+1} belongs to J_{k+1} (Lemma 3.3); otherwise the branch is discarded.

A valid branch produces a successor state $(J_{i,k+1}, A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}})$ where $J_{i,k+1}$ extends the previous $J_{i,k}$ with the newly formed $S_{k+1} = U$. It is inserted into $\mathcal{Y}^{(k+1)}$ under the dominance rule: among states with the same block partition $(S_1, \dots, S_{k+1})$, only the one with minimal z_{cur} for each $(A_{\text{cur}}, B_{\text{cur}})$ is retained.

The complete procedure is given in Algorithm 1.

Algorithm 1 DP_F2_WeightedSum

Require: Jobs partitioned into $J_1, \dots, J_K$ with $W_1 > \dots > W_K$; each L_k is the Johnson-ordered list of J_k .

Ensure: optimal weighted sum Z^*

```
1:  $\mathcal{Y}^{(0)} \leftarrow \{(\emptyset, 0, 0, 0)\}$  ▷ no block built yet
2: for  $k = 0$  to  $K - 1$  do
3:    $\mathcal{Y}^{(k+1)} \leftarrow \emptyset$ 
4:   for each  $(J_{i,k}, A, B, z) \in \mathcal{Y}^{(k)}$  do
5:     Merge the remaining suffixes of  $L_{k+1}, \dots, L_K$  in Johnson order into a sequence  $\sigma$ 
6:     Initialize a stack with  $(\ell, A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}}, U, \text{last}) \leftarrow (1, A, B, z, \emptyset, \text{nil})$ 
7:     while stack is not empty do
8:       Pop  $(\ell, A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}}, U, \text{last})$ 
9:       if  $\ell > |\sigma|$  then
10:        if  $U \neq \emptyset$  and  $\text{last} = k+1$  then
11:          Let  $S_{k+1} \leftarrow U$ ;  $J_{i,k+1} \leftarrow J_{i,k} \cup \{S_{k+1}\}$ 
12:          Advance the taken-prefix pointer of each  $L_m$  by  $|U \cap J_m|$ 
13:          if  $\nexists (J_{i,k+1}, A_{\text{cur}}, B_{\text{cur}}, \hat{z}) \in \mathcal{Y}^{(k+1)}$  with  $\hat{z} \leq z_{\text{cur}}$  then
14:            Insert  $(J_{i,k+1}, A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}})$  into  $\mathcal{Y}^{(k+1)}$ , pruning dominated states
15:          end if
16:        end if
17:        continue
18:      end if
19:       $j \leftarrow \sigma[\ell]$ , let  $m$  be the class index of  $j$ 
20:      if  $m = k+1$  then
21:         $A' \leftarrow A_{\text{cur}} + a_j$ ;  $B' \leftarrow \max(B_{\text{cur}}, A') + b_j$ ;  $z' \leftarrow z_{\text{cur}} + w_j \cdot B'$ 
22:        Push  $(\ell+1, A', B', z', U \cup \{j\}, k+1)$  onto stack
23:      else
24:         $A' \leftarrow A_{\text{cur}} + a_j$ ;  $B' \leftarrow \max(B_{\text{cur}}, A') + b_j$ ;  $z' \leftarrow z_{\text{cur}} + w_j \cdot B'$ 
25:        Push  $(\ell+1, A', B', z', U \cup \{j\}, \text{last})$  onto stack
26:        Push  $(\ell+1, A_{\text{cur}}, B_{\text{cur}}, z_{\text{cur}}, U, \text{last})$  onto stack
27:      end if
28:    end while
29:  end for
30: end for
31:  $Z^* \leftarrow \min\{z \mid (J_{i,K}, A, B, z) \in \mathcal{Y}^{(K)}\}$ 
32: return  $Z^*$ 
```

5 Conclusion